



DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

PRIVACY-ENHANCED ONTOLOGIES

A STUDY ON SECURE PROTOCOLS FOR EXPRESSIVE SPARQL
QUERIES OVER ENCRYPTED ONTOLOGIES

MASTER IN COMPUTER SCIENCE

NOVA University Lisbon

Draft: February 18, 2022



PRIVACY-ENHANCED ONTOLOGIES

A STUDY ON SECURE PROTOCOLS FOR EXPRESSIVE SPARQL QUERIES OVER
ENCRYPTED ONTOLOGIES

DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

Advisers: Matthias Knorr
Assistant Professor, NOVA University Lisbon
João Leitão
Assistant Professor, NOVA University Lisbon
Henrique Domingos
Associate Professor, NOVA University Lisbon

ABSTRACT

Ontologies provide a shared understanding of a domain, by formalizing the specification of relevant concepts, and their relations, in such domain.

In general, they only contain public, and non-sensitive data, but sometimes they are used alongside electronic health records and clinical decision systems. These systems heavily manipulate personal data, which is sensitive. Additionally, ontologies themselves have an inherent private value, due to the investment of time and money in their development.

With ontologies gradually growing in dimension, the industry is currently evolving towards the use of collaborative solutions and outsourcing heavy computations, relying on cloud infrastructures or other shared computing platforms. These approaches, albeit inevitable, present new security concerns.

In this thesis, we investigate existing secure computation techniques and propose the definition of a new Symmetric Searchable Encryption protocol that supports expressive SPARQL queries over encrypted ontologies.

Keywords: Ontologies, Searchable Encryption, Secure Computing, SPARQL

CONTENTS

List of Figures	v
List of Listings	vi
Acronyms	vii
1 Introduction	1
1.1 Context	1
1.2 Problem	3
1.3 Objectives	4
1.4 Contributions	4
1.5 Document Organization	5
2 Related Work	6
2.1 Semantic Web: A layered architecture	6
2.1.1 RDF: Resource Description Framework	8
2.1.1.1 Overview	8
2.1.1.2 Syntax	8
2.1.2 RDFS: RDF Schema	9
2.1.2.1 Overview	9
2.1.2.2 RDFS primitives	9
2.1.3 OWL2: Web Ontology Language	10
2.1.3.1 Overview	10
2.1.3.2 OWL Language	11
2.1.3.3 OWL2 DL	14
2.1.4 SPARQL	15
2.1.4.1 Overview	15
2.1.4.2 SPARQL Queries	15
2.1.5 Tools & Development Software for Ontologies	17
2.2 Secure Computations	18

2.2.1	Homomorphic Encryption	18
2.2.2	Searchable Encryption	19
2.2.2.1	Overview	19
2.2.2.2	Searchable Encryption (SE) General Index-Based Model	19
2.2.2.3	Client/Server Model	20
2.2.2.4	Information Leakage	20
2.2.3	Symmetric Searchable Encryption	21
2.2.3.1	Overview	21
2.2.3.2	Security Definitions	21
2.2.3.3	Symmetric Searchable Encryption (SSE) Schemes	21
2.2.4	Other Approaches	23
2.3	Summary	24
3	Approach to Elaboration	25
3.1	A Protocol for Secure Operations over Privacy-Enhanced Ontologies	25
3.2	System Model & Architecture	26
3.3	Evaluation Guidelines	26
4	Work Plan	28
4.1	Tasks	28
	Bibliography	29

LIST OF FIGURES

2.1	The <i>Semantic Web</i> stack [104].	7
2.2	The Web Ontology Language (OWL2) Structure [85].	11

LIST OF LISTINGS

2.1	Simple RDF example, written in Terse RDF Triple Language (TURTLE) syntax.	9
2.2	Simple OWL2 example, written in TURTLE syntax.	12
2.3	Simple SPARQL Protocol and RDF Query Language (SPARQL) example	16
2.4	SPARQL query with filters and modifiers example	17
2.5	SPARQL a more complex pattern example	17

ACRONYMS

API	Application Programming Interface 18
BSBM	Berlin SPARQL Benchmark 17 , 26
FHE	Fully Homomorphic Encryption 3 , 19
GO	Gene Ontology 1
HE	Homomorphic Encryption 3 , 6 , 18 , 19 , 24
HTML	HyperText Markup Language 9
HTTP	Hypertext Transfer Protocol 17
IND-CKA2	Indistinguishability Against Adaptive Chosen Keyword Attack 21 , 22 , 25
IND-CKA1	Indistinguishability Against Non-Adaptive Chosen Keyword Attack 21 , 22
LFHE	Leveled-Fully Homomorphic Encryption 3 , 19
M/M	Multiple Writer/Multiple Reader 20 , 26
M/S	Multiple Writer/Single Reader 20
NCI	National Cancer Institute 1
ORAM	Oblivious RAM 3 , 18 , 24
OWL2	Web Ontology Language v , vi , 7 , 8 , 10 , 11 , 12 , 13 , 14 , 15 , 17 , 23 , 24 , 25 , 26
PEKS	Public Key Encryption with Keyword Searchable 3 , 19 , 24
PHE	Partially Homomorphic Encryption 3 , 19
PRF	Pseudo-Random Function 21

PRP	Pseudo-Random Permutation 22
RDF	Resource Description Language vi, viii, 7, 8, 9, 10, 11, 15, 16, 17, 23, 24
RDFS	Resource Description Framework Schema 7, 8, 9, 10, 11, 12, 24
S/M	Single Writer/Multiple Reader 20
S/S	Single Writer/Single Reader 20, 24, 26
SE	Searchable Encryption iv, 3, 6, 18, 19, 20, 23, 24, 26
SHE	Somewhat Homomorphic Encryption 3, 19
SLA	Service Level Agreement 2
SPARQL	SPARQL Protocol and RDF Query Language vi, 4, 7, 8, 15, 16, 17, 24, 25, 26
SQL	Structured Query Language 16
SSE	Symmetric Searchable Encryption iv, 3, 4, 19, 21, 23, 24, 25, 28
STE	Structured Symmetric Encryption 4, 23
SWRL	Semantic Web Rule Language 7
TURTLE	Terse RDF Triple Language vi, 8, 9, 12, 16
URI	Uniform Resource Identifiers 6, 8, 12
W3C	World Wide Web Consortium 6, 17
WWW	World Wide Web 1
XML	Extensible Markup Language 7, 9, 11

INTRODUCTION

1.1 Context

The Semantic Web In 2001, the term *Semantic Web* [7] was first used to define what would be the evolution of the [World Wide Web \(WWW\)](#). The traditional *Web* solved the challenge of distributing information at scale and globally, however it was designed for human use. Documents are easily accessible and readable by humans, but for automated agents little more than document retrieval is possible. Contrary to humans, machines cannot reason over the information they access, if they do not have access to the *semantics of the data*.

The *Semantic Web* addresses this issue, by redesigning the representation of information, in a way that not only preserves meaning and relationships but is also machine-readable. A concept is mapped to a unique identifier, and its meaning is encoded in sets of triples, each triple consisting of a subject, predicate and object (similar to an elementary sentence). *Ontologies* provide a shared understanding of a domain, by formalizing the specification of relevant concepts, and their relations, in such domain.

This novel method of knowledge representation differs from the previous, centralized way, and greatly simplifies the process of interconnecting different knowledge bases. Centralized representations tend to be more expensive to develop, usually needing to be built from scratch [48]. Furthermore, for sharing, both parties need to agree on the semantics. In contrast, using ontologies, information is semantically linked by default. In the *Semantic Web*, machines can truly communicate, share and reason about distinct knowledge bases.

Industry Landscape Knowledge representation is essential in the industry [50] and ontologies help remove ambiguity in terminology. Nowadays, they are being developed and applied in a variety of domains [98] ranging from the health sector¹, to linguistics²

¹The NCI Thesaurus [70], developed by the [National Cancer Institute \(NCI\)](#) or the [Gene Ontology \(GO\)](#) [30], the world's largest source of information on the functions of genes, are some existing applications.

²The WordNet [44] is a lexical database for English where related words and concepts are semantically linked.

and even being used by companies like NASA³ [6].

To develop and maintain ontologies, advanced semantic editors are the standard tool for professionals, with Protégé [47] being a very popular example. These provide numerous functionalities, usually based on plugins (e.g. automatic verification of errors and consistency in the design; debugging tools; query answering; graphic visualization of data and knowledge).

With ontologies gradually growing in dimension, the industry is currently evolving towards the use of collaborative solutions [81, 80] and outsourcing heavy computations, relying on cloud infrastructures or other shared computing platforms. These approaches, albeit inevitable, present new security concerns and engineering challenges.

As of 2021, cloud adoption has risen more than what was previously forecasted, with 50 percent of all enterprise workloads hosted in *public clouds* and predictions of 7 percentage point gain in 2022 [69]. Research on secure cloud computing [14] studies such adoption but also discusses security concerns. Adoption can be explained with the cost efficiency⁴, high scalability, flexibility and availability that cloud provides. Additionally, the pay-per-use model is an easy entry point to the ecosystem.

Cloud Security Concerns While delegation of responsibilities is beneficial for costs, it is prejudicial to control. Using cloud services implies trusting its owner. A [Service Level Agreement \(SLA\)](#) defines, in accordance to metrics, the service levels of the system that the vendor should provide (e.g. percentage of availability), while also defining penalties in case of not achieving what was agreed-on. However, the risk of loss of privacy and data breaches still exists. A problem that is especially concerning when sensitive data, such as health records, is being stored in such services. This is much more relevant in *public clouds* where, compared to *private clouds*, trust is distributed between various actors and not on a single organization, namely the cloud operator [14].

In cloud storage services, administrators, or hacker with root privileges, might have access to the data [10]. Additionally, these cloud services might outsource computations to third parties whom we do not trust. In this setting, we usually consider the *honest-but-curious*, or *passive*, adversary model. This adversary will follow the protocol properly but keep a record of all its intermediate computations [28]. Furthermore, it cannot do any other action except attempting to learn private information while observing views of the protocol execution [23].

Security Approaches A first approach to secure data in the cloud would be to use *Encryption at Rest*, which naively encrypts all data stored in the untrusted servers. If a user wants to do a write or search operation over the data, they will have to download all

³Observing the visual representation provided by the Linked Open Data Cloud [42] is a good exercise to perceive the dimension of adoption of this new paradigm.

⁴Cloud services reduce implementation and maintenance, power, cooling, floor space, storage and operational costs.

the data, and decrypt it. This incurs in high network requirements and is computationally expensive on the client-side, hindering all the benefits of using a cloud service to store the data. Nonetheless, this is the mechanism usually provided by database services [54, 53, 46].

Recently, active research is being done in the fields of [SE](#) and [Homomorphic Encryption \(HE\)](#).

[Homomorphic Encryption](#) is a cryptographic method based on the concept of algebra homomorphism, where both decryption and encryption can be seen as an homomorphism of the relations between the plaintext and the ciphertext [3, 26]. [Homomorphic Encryption](#), based on the type and number of operations (e.g. addition or multiplication of encrypted values) allowed to be done on the encrypted data, can be classified as: [Fully Homomorphic Encryption \(FHE\)](#) supporting any number of operations with no restriction on the number of times; [Somewhat Homomorphic Encryption \(SHE\)](#) permitting some types of operations, a limited number of times; [Partially Homomorphic Encryption \(PHE\)](#) allowing one type of operation with no restriction on the number of times; and [Leveled-Fully Homomorphic Encryption \(LFHE\)](#) [11] supporting any type of operations, but with a pre-determined expected depth- L (can execute them L times).

[Searchable Encryption](#) uses classical encryption schemes to perform encrypted search queries, sequentially over the ciphertext [72], or through the generation of searchable *encrypted indexes* accessible with *tokens*, generated via *trapdoor* functions. [Searchable Encryption](#) can be divided in [SSE](#), when it is based on symmetric key cryptography, and [Public Key Encryption with Keyword Searchable \(PEKS\)](#), when asymmetric key algorithms are used. These techniques may leak some information about the *indexes*, *search pattern*, and *access pattern* during execution. Most approaches leak, at least, the *search* and *access patterns* [10]. Recently, to mitigate information leakage, *hybrid* solutions combining the software approach of [SE](#) with *trusted hardware* have been proposed [24].

Besides [HE](#) and [SE](#), [Oblivious RAM \(ORAM\)](#) [29] could theoretically be used as an alternative. However, [ORAM](#) and [HE](#) are not efficient enough, when used exclusively, to be feasible yet [10] in real world applications. Furthermore, [SSE](#) tends to be more practical than [PEKS](#), due to the efficiency comparison between asymmetric and symmetric cryptographic schemes [58].

1.2 Problem

Usually, ontologies only contain public, and non-sensitive data, but sometimes they are used alongside electronic health records and clinical decision systems. These systems heavily manipulate personal data, which is sensitive. Additionally, ontologies themselves have an inherent private value, due to the investment of time and money in their development.

Taking into consideration the efficiency, security trade-offs and ease of implementation from the various alternative techniques, a pure software [SSE](#) approach seems to be

the most practical solution for protecting ontologies in untrusted servers, not requiring excessive computational power or special trusted hardware. However, most [SSE](#) protocols only consider *unstructured* collections of documents with no semantic relationships. More recently, Chase and Kamara, on their work on [Structured Symmetric Encryption \(STE\)](#), generalized index-based [SSE](#) to support queries on *structured* data [15].

Ontologies can be represented as graphs and [STE](#) supports various types of query operations in graph-structured data. Nevertheless, a [STE](#) algorithm that supports the query expressiveness required by [SPARQL](#) [102, 100], the recommended language to query published semantic data, has yet to be implemented. Existing schemes, adapted to conceptual graphs, need to pre-compute all possible query answers *a priori*, which is not practical for large ontologies [62]. Furthermore, the information leakage can be a problem while trying to protect the topology of an ontology, without first pre-computing all query answers.

To conclude, we will focus on answering the following questions:

1. *How to support expressive queries and operations in privacy-enhanced encrypted ontologies, using [Symmetric Searchable Encryption](#)?*
2. *Besides preserving data privacy, how can we minimize information leakage and avoid possible exposure of the topology of the encrypted ontologies?*

1.3 Objectives

The main objective of this thesis will be to provide answers to the two questions defined in section 1.2, through a problem-driven approach, and guide future work in this field of research.

We will first study a static [SSE](#) scheme for structured data, focusing on supporting [SPARQL](#) queries in a multi-user setting, later expanding to a dynamic setting. Finally, we will study how to minimize information leakage about the topology of the encrypted data.

1.4 Contributions

Our contributions will be the proposal of a protocol that extends upon existing [SSE](#) schemes for structured data. The protocol supports expressive and secure [SPARQL](#) queries over privacy-enhanced encrypted ontologies. We now present a summary of the contributions:

P - A Protocol for Secure Operations over Privacy-Enhanced Ontologies A novel multi-user [SSE](#) protocol to support [SPARQL](#) queries over encrypted ontologies, stored in untrusted servers.

P1 - Static Setting The protocol implementation for a static setting, supporting only query operations.

P2 - Dynamic Setting The protocol implementation for a dynamic setting, supporting search, write and delete operations.

All the prototypes developed will be available as open-source code in a public repository, alongside the benchmark results.

1.5 Document Organization

The remaining of the document will be organized as follows:

Chapter 2 Analyses related work on *Semantic Web* and *Secure Computations*.

Chapter 3 Presents requirements and informally defines the protocol. Furthermore, in this chapter we describe our implementation and testing approaches.

Chapter 4 Describes the work plan of the elaboration phase tasks.

RELATED WORK

On this chapter, we will discuss the architecture and design of the *Semantic Web* as well as analyzing work already done in the fields of *Secure Computations*. The remaining of the chapter is organized as follows: on Section 2.1 we discuss the *Semantic Web* stack, languages and tools; Section 2.2 describes work on *Secure Computation*, with particular attention regarding HE and SE; finally, on Section 2.3 we summarize and discuss which work is relevant.

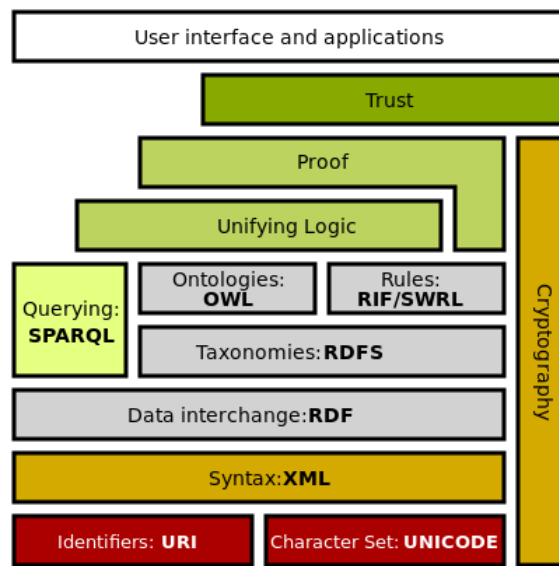
2.1 Semantic Web: A layered architecture

In Section 1.1, we have contextualized the main philosophy and design principles of the *Semantic Web*:

1. Usage of *labeled graphs* as the data model where concepts are represented as triples consisting of a subject, predicate and object (with nodes being objects and relations between such objects modelled as the edges);
2. All data items are identified by [Uniform Resource Identifiers \(URI\)](#);
3. Usage of ontologies to model semantics within a domain.

Additionally, the *Semantic Web* can also be seen as a stack of various layers [5] (figure 2.1). Most of these layers are standardized due to the collective work of the [World Wide Web Consortium \(W3C\)](#).

Briefly describing this layered architecture, at the root level, we have the [URIs](#) and a character set providing the baseline of communication, with distinguishable identification of resources and an agreed encoding.

Figure 2.1: The *Semantic Web* stack [104].

At the syntax level we find the [Extensible Markup Language \(XML\)](#) [83]. XML supports the definition of unlimited *tags*, that can be placed in the text of a document. These *tags* facilitate the creation of hierarchy and give *structure* to the information.

From structured data we then define the basic data model using [RDF](#) [92]. With a XML like syntax, we can write statements about resources, thus embedding *semantics* within the information, as *metadata*.

The [Resource Description Framework Schema \(RDFS\)](#) [96] is a vocabulary description language to organize [RDF](#) resources into hierarchies, based on primitives such as *classes* and *properties*. It can be seen as a rudimentary *ontology language* [5].

Ontologies are defined with [OWL2](#) [85]. Like [RDFS](#), it is also a vocabulary description language, but provides more expressiveness (e.g. relations between classes, *cardinality*, *equality*, richer properties with characteristics).

[RDF](#) triples can be interpreted as logical binary predicates, making reasoning possible over [RDF](#)-based data. This allows us to infer and validate ontologies, and also provide explanations on operations' results. The rules layer is a reference to this specific part of the architecture stack. There exist various languages, such as the [Semantic Web Rule Language \(SWRL\)](#) [103], that allow us to express rules as well as logic.

To query and retrieve any data based on [RDF](#) we use [SPARQL](#).

The Proof and Unifying Logic layers include the notion of interconnectivity of terminologies between *knowledge bases*, with explainable proofs to operations.

Only with secure cryptographic schemes can we reach the trust layer. The security layer is still not standardized.

The top layer, references all the applications and interfaces that use this new paradigm of data representation.

In the following of this Section we will describe, in more detail, some languages and

tools. We will focus on [RDF](#) in Section 2.1.1; on [RDFS](#), in Section 2.1.2; on [OWL2](#), in Section 2.1.3; on [SPARQL](#), in Section 2.1.4; on common tools and software, in Section 2.1.5.

2.1.1 RDF: Resource Description Framework

2.1.1.1 Overview

[RDF](#) [92] is a flexible *domain-independent* data model. The [RDF](#) basic primitives are *resources*, *properties* and *literals*. We can construct *statements* about resources by organizing these basic primitives in sets of *triples*. A triple is composed of a *subject*, a *predicate* and an *object*. In a statement (triple) about a subject (always a resource) we define the value (object) of a property (predicate), which can be another resource or a literal.

Literals are atomic values (e.g. strings, numbers). Resources and properties, respectively, model the "things" and relationships between such "things". Both are identified using [URIs](#). Resources without a [URI](#) are called *blank nodes* (or *bnodes*).

To make a statement *B* about a statement *A* we can use a technique called *reification*. Reification consists in: firstly, defining a statement *B* where the object *objA* represents the statement *A*; secondly, creating the properties *subject*, *predicate* and *object* mapped from *objA* to the original subject, predicate (now a resource), and original object of statement *A* [5].

2.1.1.2 Syntax

Graphical [RDF](#) being a data model, can be written using different syntaxes. Graphically we can represent statements as a labeled and directed graph. The *nodes* are labelled with the resources [URIs](#), a literal value or left blank, whereas *arcs* are labelled with the properties [URIs](#). The arcs are directed from subject to object of a statement.

1) TODO: ADD graphical example

T¹

Text-Based There are multiple text-based syntaxes for [RDF](#). Some of the most commonly used are [TURTLE](#) [94] (listing 2.1), and its extension for named graphs *TriG* [93]. In [TURTLE](#), [URIs](#) are enclosed by angle brackets. The subject, property, and object are written in this order. Statements are finished by a period. Literals are surrounded in quotes with the type of data specified with a [URI](#). The data value and type are separated by ^^ (if no data type is mentioned, the value is inferred as being a string). We can also define [URI](#) abbreviations (using @prefix, followed by the prefix to substitute the [URI](#), a colon, the [URI](#) and a period) and simplify statements, for easier readability (e.g. using a semicolon to make more than one statement about the same subject, defining statements with nested blank nodes using square brackets). Comments start with #.

```

1 # FCT is located in Almada
2 <http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/ontology/
   ↳ location> <http://www.my-custom-domain/resource/Almada> .
3
4 # FCT has 8.000 students
5 <http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/faculties.
   ↳ ttl#asNumberOfStudents> "8000"^^<http://www.w3.org/2001/XMLSchema#integer> .

```

Listing 2.1: Simple RDF example, written in **TURTLE** syntax.

XML/RDF [95] is a standardized **XML**-based syntax and *RDFa* [97] permits **RDF** to be embedded in the **HyperText Markup Language (HTML)**¹.

2.1.2 RDFS: RDF Schema

2.1.2.1 Overview

RDF is a flexible data-model. However, for interconnectivity of different **RDF** knowledge bases, semantics for restricting meaning in a specific domain are needed.

RDFS [96] does that by providing vocabulary to describe groups of related resources and the relationships between these resources. This vocabulary is a set of **RDF** resources identified under the <http://www.w3.org/2000/01/rdf-schema#>.

2.1.2.2 RDFS primitives

Classes group resources together, the *instances* of the class. *Properties* are defined by specifying to which classes of resource they may apply, using the mechanism of *domain* and *range*. This allows for easy extension, by not having to redefine classes wherever a new property, that applies to an existing class, is needed.

To state that a resource is an instance of a class we use the *rdf:type* property.

The *rdfs:subClassOf* is used to define that all the instances of a class are instances of its *superclasses*: if *A* *rdfs:subClassOf* *B*, then instances of *A* are also instances of *B*. The *rdfs:subPropertyOf* states that instances related by a property are also related by its *superproperties*: if *A* *rdfs:subPropertyOf* *B*, then instances related to *A* are also related to *B*. The properties *rdfs:subClassOf* and *rdfs:subPropertyOf* are *transitive*, and there are no restrictions on the number of superclasses or superproperties that, respectively, a class or a property can have.

The *rdfs:domain* property defines that any resource with a given property is an instance of the domain classes that the property applies to. The *rdfs:range* property specifies that the values of the property are instances its range classes.

In tables 2.1 and 2.2, we summarize the core **RDFS** classes and properties. In figure ..., we extend the **RDF** graphical example ... with **RDFS**.

T²

¹We recommend the curious reader to read the W3C recommendations to better learn the syntaxes.

2) TODO: ADD graphical example

RDFS Core Classes	
Class	Description
rdfs:Resource	The class resource, everything.
rdfs:Class	The class of classes.
rdfs:Literal	The class of literal values, e.g. textual strings and integers.
rdf:Property	The class of RDF properties.
rdfs:Statement	The class of RDF statements.

Table 2.1: RDFS Core Classes and respective description (rdfs:comment).

Core RDFS Properties			
Property	Description	Domain	Range
Relationship Definition			
rdf:type	The subject is an instance of a class.	rdfs:Resource	rdfs:Class
rdfs:subClassOf	The subject is a subclass of a class.	rdfs:Class	rdfs:Class
rdfs:subPropertyOf	The subject is a subproperty of a property.	rdf:Property	rdf:Property
Property Restriction			
rdfs:domain	A domain of the subject property.	rdf:Property	rdfs:Class
rdfs:range	A range of the subject property.	rdf:Property	rdfs:Class
Reification			
rdf:subject	The subject of the subject RDF statement.	rdf:Statement	rdfs:Resource
rdf:predicate	The predicate of RDF statements.	rdf:Statement	rdfs:Resource
rdf:object	The object of RDF statements.	rdf:Statement	rdfs:Resource
Utility			
rdfs:seeAlso	Further information about the subject resource.	rdfs:Resource	rdfs:Resource
rdfs:isDefinedBy	The definition of the subject resource.	rdf:Resource	rdfs:Resource
rdfs:comment	A description of the subject resource.	rdf:Resource	rdfs:Literal
rdfs:label	A human-readable name for the subject.	rdf:Resource	rdfs:Literal

Table 2.2: RDFS Core Properties and respective description (rdfs:comment).

2.1.3 OWL2: Web Ontology Language

2.1.3.1 Overview

An *ontology* was defined by Studer et al. as "a formal, explicit specification of a shared conceptualisation" [74]. An ontology should specify consensual and machine-readable knowledge by means of a formal definition of concepts, where concepts have explicit types and restrictions. Therefore, an *ontology language* should provide a well-defined syntax, formal semantics, convenient and sufficient expressiveness and efficient reasoning support.

RDFS can be considered a very limited *ontology language*. OWL2 [85] provides a number of additional features (e.g. cardinality constraints, class equivalence, intersection and disjunction). OWL2 does not simply extend RDFS, because RDFS primitives are too expressive. This would make reasoning undecidable [5]. As such, OWL2 is divided in two sublanguages: OWL2 Full [89], a direct extension of RDFS with all OWL2 semantics; OWL2 DL [84], a language with some restrictions on the way OWL2, RDF and RDFS primitives may be used.

The correspondence theorem of [84] states that: given an *OWL2 DL* ontology, inferences drawn using its *Direct Semantics* will still be valid if the ontology is mapped into an RDF graph and interpreted using the *RDF-Based Semantics*, the semantics of *OWL2 Full*.

OWL2 can be expressed with any valid *RDF* syntax, however there are various specific syntaxes: the Functional-Style Syntax [90], which is very compact and was the language used in the formal specification of the language; the Manchester Syntax [86], designed to be human-readable (it is very popular in user interfaces of semantic editors); and the *OWL/XML* Syntax [91], which allows easier processing with *XML* tools.

Figure 2.2 details the structure of *OWL2*. Finally, *OWL2* can have different specifications, called profiles [87], which provide reasoning in polynomial time. The coverage of *OWL2* profiles is not on the scope of this work.

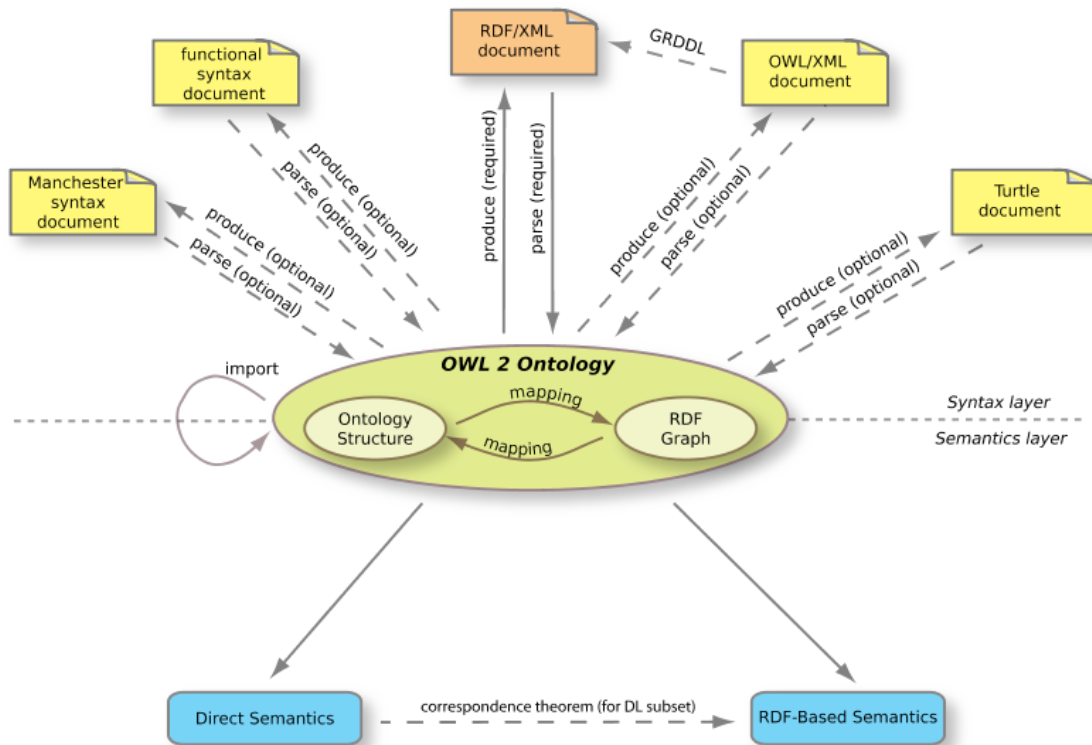


Figure 2.2: The *OWL2* Structure [85].

2.1.3.2 OWL Language

In *OWL2* the term individual is equivalent to instance in *RDFS*. An *assertion* is a statement about a resource. *Expressions* are a combination of classes, properties and instances. An *axiom* relates such expressions to classes². Listing 2.2 provides a simple *OWL2* example.

²For an overview of the language we recommend the reader to consult the reference guide [88].

```
1 @prefix owl: <http://www.w3.org/2002/07/owl#> .
2 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
3 @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
4 @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
5
6 # The base namespace of the FCT ontology
7 @base<http://www.my-custom-domain/fct.ttl> .
8
9 <http://www.my-custom-domain/fct.ttl>
10   rdf:type owl:Ontology ;
11   rdfs:label "FCT_Ontology"^^xsd:string ;
12   rdfs:comment "An example OWL2 ontology"^^xsd:string ;
13   owl:versionIRI <http://www.my-custom-domain/fct.ttl#1.0> .
14
15 # Class Expression
16 _:x rdf:type owl:Class ;
17     owl:unionOf ( :Mandatory :Optional ) .
18
19 # Class Axiom
20 :Course owl:equivalentClass _:x .
21
22 # Class Assertion
23 :class101 rdf:type :Course .
```

Listing 2.2: Simple [OWL2](#) example, written in [TURTLE](#) syntax.

Individual Assertions

In [OWL2](#) *assertions* about individual resources can be used to define their type, class membership, properties and even provide ways to define identity and negative statements. More specifically on the identity statements, because [OWL2](#) has an open world assumption, we can never differentiate two individuals just because they have different [URIs](#). To do so we use `owl:differentFrom`. To explicitly state negative facts we use `owl:NegativePropertyAssertion` [5].

Classes

In [OWL2](#) every resource that is of type `owl:Class` is a class. The two classes defined by default are *owl:Thing* and *owl:Nothing*. Respectively, these two specify the superclass of all classes and the subclass of all classes: every individual is a member of `owl:Thing` and no individual can be member of `owl:Nothing`. This is very useful for reasoning, because inconsistency is detected when classes are equivalent to `owl:Nothing` (have no individuals in their membership).

Class Axioms In [OWL2](#) we can relate *expressions* to classes using *axioms*. These can define:

- *Subclass Relations*, with equivalent usage like in [RDFS](#);
- *Class Equivalence*, in equivalent classes every individual must be a member of each class;

- *Enumerations*, a type of class definition via explicitly stating all individuals who belong to the class membership;
- *Intersection*, in a class described as the intersection of various classes, all of its members must be also individuals belonging to the membership of each of those classes;
- *Union and Disjoint Union*, in a class described as the union of various classes, every individual belonging to its membership must also belong to the membership of at least one of those classes. In the particular case of disjoint union, the intersection of the classes is equivalent to owl:Nothing;
- *Disjoint Classes*, in disjoint classes no individual can be a member of both class (the intersection of the classes is equivalent to owl:Nothing);
- *Complement*, complement classes contain all the individual such that the union of both classes is equivalent to owl:Thing.

Properties

OWL2 has three types of properties: *object* properties, which relate individuals to other individuals; *datatype* properties, which relate individuals to literals; and *annotation* properties; which are used to specify human-readable labels, comments and explanations.

The *top* and *bottom* properties specify, respectively, properties that relate to all and no individuals: owl:topObjectProperty is the superproperty of all owl:ObjectProperties and owl:bottomObjectProperty relates to no individual; owl:topDataProperty is the superproperty of all owl:DatatypeProperties, relating an individual to any possible literal value, and owl:bottomDataProperty relates no individual to any literal value.

Properties can have different characteristics:

- *Transitive Properties*, if a relates to b via property p and c relates to a via property p , then c relates to b via property p ;
- *Symmetric Properties*, if a relates to b via property p , then inverse is true;
- *Asymmetric Properties*, if a relates to b via property p , then the inverse is $\perp$;
- *Functional Properties*, if for each instance the property can only have one unique value;
- *Inverse-functional Properties* if the object of a property statement relates to a single subject;
- *Reflexive Properties*, if every individual relates to itself via the property;
- *Irreflexive Properties*, if no individual relates to itself via the property.

Property Axioms Just like class axioms, we can specify *property axioms* to describe the way properties relate to classes and other properties. These allow us to define:

- *Domain and Range*, used to determine class membership of individuals;
- *Inverse Relationship*, used to specify the inverse of the property;
- *Equivalence*, used to specify equivalent properties. Individuals related by one property will always relate by all equivalent properties;
- *Disjointness*, used to specify disjoint properties, where individuals related by one property can never be related by the other;
- *Property Chains*, used to aggregate connected properties under another property.

More Granular Class Axioms

We can write more granular class definitions in [OWL2](#) than by simply defining class axioms. This is achieved by combining class and property axioms. It is done by defining a class axiom that relates to an anonymous class (owl:Restriction) which contains axioms on its properties. The anonymous class definition captures all individuals that satisfy such restrictions. As such we can define:

- *Universal and Existential Restrictions* which the members of the class must satisfy. The class is defined as a subclass of an anonymous class with property restrictions on, respectively, owl:allValuesFrom and owl:someValuesFrom;
- *Necessary and Sufficient Conditions*, relating a class to a property, using rdfs:subClassOf, does not provide sufficient information to conclude whether an individual belongs to the class membership or not (it only states necessary, not sufficient, conditions for class membership). With owl:equivalentClass, it becomes clear that an individual belongs to the class membership, if it satisfies the restriction (the conditions are necessary and sufficient).
- *Value, Cardinality, Data Range and Datatype Restrictions* on the anonymous class, limiting the value, cardinality of the value, range of possible values and the datatype of the property value relating to members of the superclass;
- *Self Restrictions* on the individuals captured by the anonymous class.

2.1.3.3 OWL2 DL

Contrary to *OWL2 Full*, *OWL2 DL*, being less expressive, retains *decidability*. In this Section, we specify the restrictions imposed by *OWL2 DL* [5]:

- The application of [OWL2](#) primitives to each other is not allowed; only classes and non-literals resources may be defined, with all classes being individuals of `owl:Class`;
- Properties are disjoint individuals of either `owl:ObjectProperty` or `owl:DatatypeProperty`;
- Annotation properties are ignored by reasoners;
- Properties, for which range includes non-literal resources, are separated from those which relate to literal values;
- A resource can only be a class, property, or instance at the same time. Classes, properties or instances with the equal names will be treated as distinct (a mechanism called punning);
- Only functional properties can be assigned to datatype properties;
- Only object properties can have an inverse;
- Property chains can only involve object properties;
- Self restrictions on datatype properties are not allowed.

2.1.4 SPARQL

2.1.4.1 Overview

In the previous Section we've discussed the structure of [OWL2](#). [OWL2](#) documents have a direct mapping to [RDF](#) documents. As such, we can publish and query [OWL2](#) ontologies using the standard approach for [RDF](#) data.

[RDF](#) triples are stored in *triple stores*, specialized databases for this type of data. The underlying architecture of such databases varies, from relational to NoSQL approaches [19, 45]. The recommended query language to use is [SPARQL](#) [100]. It provides a range of operations over published [RDF](#) graphs, accessible via SPARQL endpoints: from read queries [102] to write operations [101, 99].

2.1.4.2 SPARQL Queries

Mathematical Basis An isomorphism of two graphs is an adjacency preserving bijection between their vertex sets [43].

The basis of [SPARQL](#) queries is finding matches, on triple patterns, also referred as *Basic Graph Pattern*, in a [RDF](#) graph [39, 19]. This problem is equivalent to the subgraph isomorphism problem, a generalization of the graph isomorphism problem: given two graphs G_1 and G_2 , we try to find if G_1 contains a subgraph that is isomorphic to G_2 . In the context of [SPARQL](#) queries, G_1 is the published [RDF](#) graph and G_2 is the query triple pattern.

Syntax In a [SPARQL](#) query we define variables, which will be mapped onto the values of the corresponding matching triples. We call the result set for each variable, the *bindings* of that variable. *SPARQL* queries have a similar structure to the [Structured Query Language \(SQL\)](#) queries: we define the operation to execute using a keyword, followed by the projection of variables to be shown in the result, the keyword *WHERE*, the triple pattern to match, enclosed in brackets, and, optionally, some modifiers of the result. The [SPARQL](#) syntax is very similar to [TURTLE](#), so we can also specify prefixes for namespaces.

The basic [SPARQL 1.1 Query Language](#) [102] provides the following query operations:

- *SELECT*, used to extract values from a [RDF](#) graph. The result is a table with the variable bindings;
- *ASK*, which returns a boolean, true if there was a match in the [RDF](#) graph, false otherwise;
- *CONSTRUCT*, which returns a set of triples from the constructions of a new [RDF](#) graph, based on the matching pattern;
- *DESCRIBE*, which usually return a set of [RDF](#) triples that describes the matching patterns. The results depend on implementation [61].

In listing 2.3 we show a simple query. However, [SPARQL](#) provides additional features for more complex queries (listings 2.4 and 2.5): definition of the disjunction and conjunction of triple patterns, using the keywords *UNION* and *AND*; definition of optional patterns, using the *OPTIONAL* construct; filtering bindings, using the *FILTER* construct; elimination of duplicates with *DISTINCT*; application of aggregate and various functions to variables; matching regex patterns; ordering of results with *ORDER BY*; and, usage of named graphs, with the *FROM* and *FROM NAMED* constructs.

```
1 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
2 @prefix fct: <http://www.my-custom-domain/fct.ttl>
3
4 # Operation and variables
5 SELECT ?opt_course ?man_course ?popular_course
6 # Query Pattern
7 WHERE
8 {
9     ?opt_course rdf:type fct:Optional .
10    ?man_course rdf:type fct:Mandatory .
11 }
```

Listing 2.3: Simple [SPARQL](#) example

```

1 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
2 @prefix fct: <http://www.my-custom-domain/fct.ttl>
3
4 # Operation and variables
5 SELECT DISTINCT ?popular_course
6 # Query Pattern
7 WHERE
8 {
9     ?popular_course fct:hasStudentEnrolled ?num_students
10     FILTER (?num_students > 60).
11 }
12 # Optional Modifiers
13 LIMIT 20

```

Listing 2.4: SPARQL query with filters and modifiers example

```

1 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
2 @prefix fct: <http://www.my-custom-domain/fct.ttl>
3 @prefix nova: <http://www.my-custom-domain/nova.ttl>
4
5 # Operation and variables
6 SELECT DISTINCT ?optional_course
7 # Query Pattern
8 WHERE
9 {
10     {?optional_course rdf:type fct:Optional}
11     UNION
12     {?optional_course rdf:type nova:Elective}
13 }

```

Listing 2.5: SPARQL a more complex pattern example

Besides read operations, *SPARQL Update*, provides additional options for inserting, publishing and deleting data in triple stores. For operations over the [Hypertext Transfer Protocol \(HTTP\)](#), *SPARQL 1.1 graph store HTTP protocol* has similar, but more limited, features³.

2.1.5 Tools & Development Software for Ontologies

To develop and maintain ontologies, professionals use advanced semantic editors. *Protégé* [47] is a free, open-source, and usual choice. It provides a rich ontology editing environment with full support for [OWL2](#), available in a desktop version or on the web [80]. Many of its functionalities are based on plugins: automatic verification of errors and consistency in the design using reasoners like *Pellet* [71]; debugging tools; query answering; collaborative development [81]; graphic visualization of data and knowledge.

To store RDF data, popular storage solutions are: *MarkLogic* [], *Virtuoso* [52], *Amazon Neptune* [4], *GraphDB* [51], *Apache Jena - TDB* [79]. While *Apache Jena - TDB* is primarily a [RDF](#) store, the rest are *multi-model* database systems. To test their capability to handle [SPARQL](#) queries, the [Berlin SPARQL Benchmark \(BSBM\)](#) is a standard benchmark [16].

³Full examples and syntax can be found in the [W3C](#) recommendations.

Finally, for developers, many [Application Programming Interface \(API\)s](#) [55] or full framework solutions, such as *Apache Jena* [78], are available for building *Semantic Web* applications.

2.2 Secure Computations

In Section 1.1, we have contextualized the motivations behind the need for secure computation when sharing ontologies over cloud or other distributed networks.

In this setting we consider the *honest-but-curious*, or *passive*, adversary model. As such an adversary will follow the protocol properly but keep a record of all intermediate computations [28] in an attempt to learn private information while observing views of the protocol execution [23]. Security concerns should take into account the operation history, data stored in the server and also the memory access pattern, as the adversary will have access to all of this information.

The *Encryption at Rest* approach encrypts all data stored in the untrusted server. To perform computations we need to download the full dataset, decrypt it, and only then can we execute operations on the data. This is too inflexible and computationally expensive.

[ORAM](#) [29] fits well the adversary model, as it allows for the concealment of memory access patterns by shuffling and continuously re-encrypting data on memory, but it is still impractical.

When used exclusively, [Homomorphic Encryption](#) [3, 26] schemes are not efficient enough to be feasible in real-world applications [10]. However, they are applied in conjunction with other security schemes, such as in some [Searchable Encryption](#) schemes [10, 15]. For this reason, in the following Sections we will discuss related work in the fields of [HE](#), in Section 2.2.1, and [SE](#), in Section 2.2.2.

2.2.1 Homomorphic Encryption

The term homomorphism, in security, was used for the first time by Rivest et al. in 1978 [66]. [Homomorphic Encryption](#) is a cryptographic method based on the concept of algebra homomorphism, where both decryption and encryption are seen as homomorphisms of the relations between the plaintext and the ciphertext [3, 26]. With [HE](#), computations done over encrypted data, are preserved after decryption of the ciphertext.

From [3], we define an encryption scheme to be homomorphic over an operation $\bullet$ if it satisfies the following equation, where E is the encryption algorithm and M the set of all possible messages:

$$E(m_1) \bullet E(m_2) = E(m_1 \bullet m_2), \forall m_1, m_2 \in M \quad (2.1)$$

Some [HE](#) schemes preserve additive properties [56], others maintain multiplicative properties [67, 22], and some conserve both additive and multiplicative properties [27]. The [HE](#) schemes that preserve both properties allow for any arbitrary operation, because

boolean circuits can be represented using only *XOR* (addition) and *AND* (multiplication) gates [3].

HE is classified in four main types:

- **Fully Homomorphic Encryption**, which supports any number of operations with no restriction on the number of operations;
- **Somewhat Homomorphic Encryption**, which permits some types of operations, a limited number of times;
- **Partially Homomorphic Encryption**, that allows for one type of operation with no restriction on the number of times;
- **Leveled-Fully Homomorphic Encryption** [11], which supports any type of operations for a limited, and expected, number of L times (referred to as depth- L).

2.2.2 Searchable Encryption

2.2.2.1 Overview

Searchable Encryption uses classical encryption schemes to perform encrypted search queries. While the first approach implemented it via a searchable ciphertext [72], most modern schemes do it using encrypted searchable indexes. **Searchable Encryption** can be divided in **SSE**, when it is based on symmetric key cryptography, and **PEKS**, when public key algorithms are used. We will focus on index-based **SE** schemes, more specifically in index-based **SSE** ones, due to their higher efficiency compared to **PEKS** approaches.

2.2.2.2 SE General Index-Based Model

A general model, for the index-based **SE** schemes was defined in [10]. Given a database $DB = (M_1, \dots, M_n)$ consisting in a set of n plaintext documents M , some data items (e.g, keywords in the documents) $W = (w_1, \dots, w_m)$ are extracted from the documents and encrypted using a key K , using an encryption algorithm Enc . The documents may be encrypted too, generating a collection of ciphertexts C , usually with a different key K' . A client generated an encrypted index I , for the database DB , using a *BuildIndex* function. The client stores the following information in an *honest-but-curious* untrusted server:

$$\begin{aligned} I &= BuildIndex_K(DB = (M_1, \dots, M_n), W = (w_1, \dots, w_m)) \\ C &= Enc_{K'}(DB) \end{aligned} \quad (2.2)$$

To perform search queries, the client generates a trapdoor $T = Trapdoor_K(f(w_j))$, for a predicate f a keyword w_j . With trapdoor T , the server performs the search over I , using a *Search* algorithm, by checking for an encrypted index $i = Search(I, T)$. If there is a keyword w that satisfies the predicate f , it returns to the client the corresponding ciphertexts associated with i or $\perp$ if no match was found.

T³

3) TODO: Add interaction scheme between client and server

2.2.2.3 Client/Server Model

In **SE** schemes, one or various clients, the *writers* store encrypted data in a server. To issue a query request to the server, one or multiple clients, the *readers*, generate trapdoors. There are four possible **SE** scheme architectures:

- Single Writer/Single Reader (S/S);
- Single Writer/Multiple Reader (S/M);
- Multiple Writer/Single Reader (M/S);
- Multiple Writer/Multiple Reader (M/M).

The **S/S** architecture is usually used for *data outsourcing* and **M/S**, **S/M** and **M/M** employed in *data sharing* scenarios [10]. Symmetric key schemes are typically **S/S**, while public key schemes **M/S** (multiple users can write using a *public key*, but only the owner of the *private key* can generate trapdoors). Extension of the single reader schemes to multiple reader schemes can be done with *key sharing* or with the usage of *proxy re-encryption*.

In proxy re-encryption cryptography, a proxy is used to re-encrypt a ciphertext, encrypted using key K , and generates a new ciphertext, that is decrypted using key K' , where $K \neq K'$ and the decrypted plaintext remains unchanged. Proxy re-encryption is usually employed with public key cryptography [8, 31, 21], but there are some schemes that support the symmetric key setting [77, 68, 59].

2.2.2.4 Information Leakage

Almost all **SE** techniques have privacy issues because they leak some information [10]. This can leak *index* information, *search pattern*, and *access pattern* during execution:

- *Index Information* refers to the information about the index keywords. It may include the number of keywords per document or database, the total of documents, document similarity, their length and metadata;
- *Search Pattern* includes the information about keywords, and predicates, that is inferred by correlating equal results from different searches. The search pattern can be targeted using statistical analysis;
- *Access Pattern* includes the information that may be derived from the query results, based on the memory accesses needed to retrieve the documents. (e.g. one can infer whether a query is more restrictive than another, based on the total of documents retrieved; or if some files are more accessed than others).

2.2.3 Symmetric Searchable Encryption

2.2.3.1 Overview

SSE is a type of **SSE** based on symmetric key cryptography. We will first review the standard security definitions for **SSE** and then focus on describing a few reference schemes.

2.2.3.2 Security Definitions

In 2006, Curtmola et al. [20] revisited the security definitions for **SSE** schemes, contributing with two new definitions, which are still the standard in use. Both require that only the outcome of search patterns may be leaked. Additionally, they guarantee that trapdoors do not leak information about the keywords, apart from what can be inferred from the search and access pattern. **Indistinguishability Against Non-Adaptive Chosen Keyword Attack (IND-CKA1)** and **Indistinguishability Against Adaptive Chosen Keyword Attack (IND-CKA2)** differ from each based on whether the adversary can produce an adaptive or non-adaptive attack.

- **IND-CKA1** guarantees security if the client generates all queries at once;
- **IND-CKA2** guarantees security even if the client chooses the queries based on previously obtained trapdoors and search outcomes.

2.2.3.3 SSE Schemes

Song et al. In 2000, the first practical **SSE** scheme was proposed [73]. The general basis of their method is to encrypt each word of a document and embed hash values with a specific format in the ciphertext. During search, the server executes a *sequential scan* of the document, extracts the hashes and checks for matches on their values.

First, the plaintext is divided into *fixed-size* blocks w_i and each one is encrypted with a *deterministic*⁴ encryption algorithm Enc , using key k . The ciphertexts $X_i = Enc_k(w_i)$ are split into two parts $X_i = \langle l_i, r_i \rangle$. Using a *stream cipher*⁵ as a **Pseudo-Random Function (PRF)**, a pseudo-random string of bytes $S_i = PRF_{k'}(i)$ is generated from the index position using key $k' \neq k$. Then, a hash $Y_i = \langle S_i, F_{k_i}(S_i) \rangle$ of this string of bytes is computed using a keyed hash function F_{k_i} , where the key is the output of a pseudo-random function $k_i = PRF_{k''}(l_i)$ that takes l_i as an input, with key $k''' \neq k''$. Finally, we compute the XOR of X_i with S_i to generate the final ciphertexts $C_i = X_i \oplus Y_i$.

To search, we generate trapdoor $T = \langle X, K_s \rangle$, where $X = E_k(w) = \langle L, R \rangle$ is the encryption of keyword w and $K_s = PRF_{k''}(L)$. A server searches for the keyword in a document by iterating over all ciphertexts C_i , computing $C_i \oplus X$ and checking if the output is of the form $\langle S, PRF_{K_s}(S) \rangle$ for some S .

T⁴

⁴Deterministic encryption leaks message equality.

⁵For information on stream ciphers we recommend reading chapter 8.4 of [58].

4) *TODO*: Add visual scheme

A security analysis of the scheme reveals that it leaks keyword position in the document. It is also susceptible to statistical attacks (e.g. after various searches, the words in the document can be inferred).

Curtmola et al. In 2006, two new schemes, for non-adaptive and adaptive adversaries, were proposed [20]. Each one is, respectively, at least **IND-CKA1** and **IND-CKA2** secure. The idea behind these was to use an *inverted index*, which is an index per distinct word in the database, not per document.

Non-Adaptive Scheme Considering a database with n documents, we define two data structures: an array A , where each position of the array contains a node $n_{i,j}$ which belongs to the linked list L_i associated to the keyword w_i ; a look-up table T to identify the first node of a linked list associated with a keyword.

To build A , we start by constructing the nodes. These nodes are encrypted, stored in random order (using permutations) and augmented with information to find and decrypt the next node in the list. The augmentation step happens before the encryption of the node. Each node $n_{i,j} = \langle id_m, k_{next}, p_{next} \parallel \emptyset \rangle$, includes information about the id of document m ($0 \leq m \leq n$) which contains the keyword, a randomly generated key used to encrypt the next node and a pointer to the next node in the list, or $\emptyset$ if it is the last one. The randomized disposition of the nodes, hides the length of the lists.

The look-up table I associates each keyword w_i with the node $n_{i,0} = \langle k_{i,1}, p_{i,1} \rangle \oplus PRF_k(w_i)$ stored in a random position using a **Pseudo-Random Permutation (PRP)**, $PRP_{k'}(w_i)$, where $k \neq k'$. This node contains the encryption key and the pointer to the first node of the linked list L_i .

To search, we generate trapdoor $T_w = \langle PRF_k(w), PRP_{k'}(w) \rangle$. The server uses the trapdoor to find the correct node $n_{i,0} = T[PRP_{k'}(w)]$ and then retrieve all the identifiers of the documents that contain the keyword.

Adaptive Scheme Considering a database with n documents, we now define only a look-up table I , but instead of generating the index for a single keyword w , we generate it for the family of the keyword. The family of a keyword is defined as a set of labels $F_w = \{w|str(j) : 1 \leq j \leq t\}$ where j is an integer converted to string, using str , t is the number of identifiers of documents containing w , and the label is the result of concatenating w with j . The look-up table associates each label with the identifier of a document containing the w , in a random position, using $PRP_{k'}(w|str(j))$. To hide differences in the number of labels per keyword, extra entries are padded.

To search, we generate the trapdoor $T_w = (t_1, \dots, t_n) = (PRP_k(w|str(1)), \dots, PRP_k(w|str(n)))$. The server uses the trapdoor values to find valid index positions in I and outputs the set of identifiers.

T⁵

5) TODO: Add visual scheme

Generalization to the Dynamic Setting The previous schemes were considered static, as they do not allow modifications on the database after setting up the scheme. In 2012,

Kamara et al. [33] proposed a *dynamic* extension to the *non-adaptive* scheme in [20]. They used a *deletion* array to track the positions on the search array that needed to be modified, after an update. They encrypted the array pointers, using homomorphic encryption, to support modifications on their values without decryption.

Generalization to Structured Data In 2010, Chase and Kamara [15] proposed a new adaptively secure scheme based on the *non-adaptive* scheme in [20]. This new scheme supports arbitrarily-structured data. **Structured Symmetric Encryption** is based on generating an inverted index in the form of a padded and permuted dictionary. They associated data items with metadata, allowing information about structure to be preserved if needed.

Generalization to Graph-Structured Data After **STE**, additional research has been conducted to support operations on graph-structured data.

Some authors extended **STE** to support queries on conceptual graphs [62], but they need to compute all possible query answers *a priori*.

Others, proposed a **SSE** scheme that supports *subgraph isomorphism queries* [12]. In their approach, they extract features from the graphs and generate the encrypted-indexes from the extracted features. This scheme was planned for generic graphs.

Finally, in [75] they combine an index-based **SSE** with a bucketization algorithm. By doing so, queries do not leak information about the structure of the graph. Although the authors state that their approach works with any kind of query, they only focused on neighbor and adjacency ones.

2.2.4 Other Approaches

Besides the previous **SSE** approaches, some authors proposed using onion-encryption, based on the mechanisms of *CryptDB* [65, 64, 63], for protecting graph-structured data [2].

Additionally, some research has been done for protecting individuals in **OWL2** ontologies [49], protecting only their values, leaving the schema of the ontology public. Our approach aims to fully secure the individuals and schema information.

For mitigating information leakage, *hybrid SE* schemes, that require *trusted hardware*, have been proposed [24], but solutions based on special hardware are not on the scope of this thesis.

Finally, there exists related research in specific security fields regarding **RDF** graphs.

One of those fields is ensuring *integrity* and *authenticity* of data. This has been widely studied with various signage schemes [13, 82, 34, 35, 36, 76] and one service-oriented approach [41] having been proposed.

The other field is *access control*. Various schemes have been studied and most try to embed the data with the policies, using annotations. This allows for high-granularity in the access control, which can be inferred, but requires conflict resolution [32, 1, 25, 57, 17, 38, 18]. A more simple approach is token-based [37]. As they are not the main

focus of this thesis, being only complementary work on the areas of security and ontology-engineering, we will not elaborate further on these.

2.3 Summary

In Section 2.1, we described the layered architecture of the *Semantic Web* and discussed the standard languages for describing its *data model* and *ontologies*. More specifically, we covered the semantics and syntax of **RDF** and explained the primitives of **RDFS**.

Additionally, we specified the structure of **OWL2**, focusing on the mechanism it provides to write ontologies. We also discussed the restriction imposed by *OWL DL* to remain decidable.

Furthermore, we wrote about **SPARQL**, discussing its mathematical basis, describing its syntax and enumerating its various types of queries. In an additional note, **SPARQL** has been widely studied in the literature. Research has been conducted on its semantics and complexity [60], on efficient processing of its queries [40] and on its patterns of use [9, 61]. As far as we know, no work has yet been done on its usage with encrypted **RDF** data.

We finished Section 2.1 by explaining tools and software that is used in this field of research.

In Section 2.2, we discussed various solutions to execute secure computations in untrusted servers. We started by describing the *honest-but-curious*, or *passive*, adversary model and explained why *Encryption at Rest* and **ORAM** are not practical approaches.

We proceeded to cover **Homomorphic Encryption** schemes, informally mentioning the mathematical basis of **HE** and enumerating the type of schemes available.

In Section 2.2.2, we explained the difference between **SSE** and **PEKS**. We then described the general model for index-based **SE**, as well as the *Client/Server* architecture of **SE** systems. We continued by discussing how to extend **S/S** architectures, and covered the types of privacy leakage that schemes typically have.

In Section 2.2.3 we specified the standard security definitions in use, and gave an in depth explanation of reference **SSE** schemes. We finished the **SSE** Section by showing recent generalizations, focusing on dynamic solutions and schemes that support structured data.

In Section 2.2.4, we mentioned approaches that try to secure ontologies but are not based on **SE**. Additionally, we briefly covered parallel work on security and ontologies in the fields of data *integrity*, *authenticity* and *access control*.

To conclude, we state that there still needs to be conducted more research in the field of **SSE** for graph-structured data. To justify our statement, we argue that approaches are either too generic [12], lack schema protection [49], have been implemented considering types of queries not used by **SPARQL** [75], or require pre-computation of all possible query answers beforehand [62].

APPROACH TO ELABORATION

On this chapter we will present our approach to the elaboration phase. The remaining of the chapter is organized as follows: in Section 3.1, we introduce the proposed solution; in Section 3.2 we will define the system model and architecture; in Section 3.3 we present the expected testing and validation scenarios.

3.1 A Protocol for Secure Operations over Privacy-Enhanced Ontologies

As an answer to the lack of solutions for the problems exposed in Section 1.2, we propose a new protocol that extends upon existing SSE schemes for graph-structured data. This protocol will be designed taking into consideration the necessary privacy and system requirements that exist when working with OWL2 ontologies:

Privacy Requirements Ontologies are used in various different domains, as such, they might contain sensitive information. An appropriate solution must be a SSE scheme at least IND-CKA2 secure.

System Requirements The solution should be *transparent* and as *efficient* as possible. Working with encrypted ontologies must not affect a user's workflow in any significant way. Furthermore, it should also be *flexible* and *modular*, so that it supports future improvements or modifications if needed.

Finally, professionals must still be able to execute the majority of operations that they would be able to, with normal ontologies, even if they are working with encrypted ontologies:

- **P1 - Static Setting** In this setting the protocol will only support SPARQL queries, for read operations only (*SELECT*, *ASK*, *CONSTRUCT*, *DESCRIBE*). Focusing strictly on preserving query expressiveness.
- **P2 - Dynamic Setting** In this setting, the protocol will introduce support for SPARQL queries that modify the encrypted ontology.

3.2 System Model & Architecture

System Model The system will follow the *client/server* model of [SE](#) schemes. More specifically, it will be, initially, a [S/S](#) solution, further extended to [M/M](#) using a *key sharing* approach. We will that communication channels used are *reliable* and *secure*. The system model will include a *data-owner*, *users*, an *untrusted server* and *adversaries*:

- *Data-Owner* The data-owner is the entity that owns the ontology, originally. Therefore, it is the one responsible for generating the secret keys and secure-indexes, encrypting the ontology and uploading it to a remote server. It will also be responsible for sharing the secret keys with authorized users. Additionally, the data-owner must execute *user revocation* when needed, updating the secret keys and re-encrypting the ontology.
- *Users* The users are entities who have access to the ontology and have the right secret keys. This allows them to execute secure *SPARQL* queries, over the remote encrypted ontology.
- *Untrusted Server* The untrusted server is the machine where the encrypted ontology will be stored.
- *Adversaries* An adversary is an entity that has access to the untrusted servers or possesses hold secret keys (has lost access to the ontology). The adversaries follow the *honest-but-curious* model.

6) *TODO*: Add figure

T⁶

System Architecture At a macro level, the solution will be a distributed software solution, including a *client* and a *server* component. Besides the necessary communication modules, they will include:

- *Client* The modules to encrypt [OWL2](#) ontologies, parse [SPARQL](#) queries and generate secret keys, encrypted indexes and trapdoors
- *Server* The modules to load the encrypted ontology and indexes, parse trapdoors and execute secure searches.

3.3 Evaluation Guidelines

After every implementation step we will proceed with evaluation of the prototype. The testing will follow the existing [Berlin SPARQL Benchmark](#) [16], executing the protocol in two scenarios, with the same dataset. In the first, operations will be executed with the ontology in plaintext and, in the second, with the ontology encrypted.

For evaluation, we will check performance metrics (e.g. throughput, latency). Additionally, we will also evaluate the security guarantees of the solution.

For validation, we will compare whether the output of the benchmark, using the plaintext ontology, is the same to the one, using the encrypted ontology.

WORK PLAN

On this chapter we present the work plan for the elaboration phase, depicted in the following Gantt chart —.

4.1 Tasks

During the elaboration phase we will do the following tasks:

Protocol Design which consists in the designing the [SSE](#) protocol to implement;

Implementation consists in the implementation of the protocols. At the end of this task we will have produced software artifacts;

Evaluation will be done at the end of each implementation consists in the evaluation of the prototypes in terms of performance and security, as stated in [Section 3.3](#); At the end of this task we will have produced graphs which describe the result metrics;

Validation will be done at the end of each implementation task and consists in testing the correctness of the prototyped solutions, as stated in [Section 3.3](#); At the end of this task we will have produced graphs which describe the result metrics;

Writing the rest of thesis, which will be done in parallel with the other tasks.

BIBLIOGRAPHY

- [1] F. Abel et al. “Enabling advanced and context-dependent access control in RDF stores”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 4825 LNCS. 2007, pp. 1–14. ISBN: 9783540762973. DOI: [10.1007/978-3-540-76298-0_1](https://doi.org/10.1007/978-3-540-76298-0_1) (cit. on p. 23).
- [2] N. Aburawi, A. Lisitsa, and F. Coenen. “Querying Encrypted Graph Databases”. In: *ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy*. Vol. 2018-January. 2018. DOI: [10.5220/0006660004470451](https://doi.org/10.5220/0006660004470451) (cit. on p. 23).
- [3] A. Acar et al. “A survey on homomorphic encryption schemes: Theory and implementation”. In: vol. 51. 2018. DOI: [10.1145/3214303](https://doi.org/10.1145/3214303) (cit. on pp. 3, 18, 19).
- [4] Amazon Web Services, Inc. or its affiliates. *Amazon Neptune*. <https://docs.aws.amazon.com/neptune/index.html>. Accessed: 2022-2-17 (cit. on p. 17).
- [5] G. Antoniou et al. “A Semantic Web Primer (Third Edition)”. In: *The MIT Press* (2012), p. 287 (cit. on pp. 6–8, 10, 12, 14).
- [6] N. Ashish. “Semantic-Web Technology: Applications at NASA”. In: *Dagstuhl Seminar Proceedings*. 2005 (cit. on p. 2).
- [7] T. Berners-Lee, J. Hendler, and O. Lassila. *The semantic web*. Vol. 284. 2001. DOI: [10.1038/scientificamerican0501-34](https://doi.org/10.1038/scientificamerican0501-34) (cit. on p. 1).
- [8] M. Blaze, G. Bleumer, and M. Strauss. “Divertible protocols and atomic proxy cryptography”. In: *Advances in Cryptology — EUROCRYPT’98*. Springer Berlin Heidelberg, 1998, pp. 127–144. DOI: [10.1007/BFb0054122](https://doi.org/10.1007/BFb0054122) (cit. on p. 20).
- [9] A. Bonifati, W. Martens, and T. Timm. “An Analytical Study of Large SPARQL Query Logs”. In: (Aug. 2017). arXiv: [1708.00363 \[cs.DB\]](https://arxiv.org/abs/1708.00363) (cit. on p. 24).
- [10] C. Bösch et al. “A Survey of Provably Secure Searchable Encryption”. In: *ACM Comput. Surv.* 47.2 (Aug. 2014), pp. 1–51. ISSN: 0360-0300. DOI: [10.1145/2636328](https://doi.org/10.1145/2636328) (cit. on pp. 2, 3, 18–20).
- [11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. “(Leveled) Fully Homomorphic Encryption without Bootstrapping”. In: *ACM Trans. Comput. Theory* 6.3 (July 2014), pp. 1–36. ISSN: 1942-3454. DOI: [10.1145/2633600](https://doi.org/10.1145/2633600) (cit. on pp. 3, 19).
- [12] N. Cao et al. “Privacy-preserving query over encrypted graph-structured data in cloud computing”. In: *Proceedings - International Conference on Distributed Computing Systems*. 2011. DOI: [10.1109/ICDCS.2011.84](https://doi.org/10.1109/ICDCS.2011.84) (cit. on pp. 23, 24).

- [13] J. J. Carroll. “Signing RDF graphs”. In: *Lect. Notes Comput. Sci.* 2870 (2003), pp. 369–384. ISSN: 0302-9743, 1611-3349. DOI: [10.1007/978-3-540-39718-2_24](https://doi.org/10.1007/978-3-540-39718-2_24) (cit. on p. 23).
- [14] M. Carroll, A. Van Der Merwe, and P. Kotzé. “Secure cloud computing: Benefits, risks and controls”. In: *2011 Information Security for South Africa - Proceedings of the ISSA 2011 Conference*. 2011. DOI: [10.1109/ISSA.2011.6027519](https://doi.org/10.1109/ISSA.2011.6027519) (cit. on p. 2).
- [15] M. Chase and S. Kamara. “Structured encryption and controlled disclosure”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 6477 LNCS. 2010. DOI: [10.1007/978-3-642-17373-8_33](https://doi.org/10.1007/978-3-642-17373-8_33) (cit. on pp. 4, 18, 23).
- [16] B. Chris and S. Andreas. *Berlin SPARQL Benchmark*. <http://wbsg.informatik.uni-mannheim.de/bizer/berlinsparqlbenchmark/spec/index.html>. Accessed: 2022-2-17 (cit. on pp. 17, 26).
- [17] L. Costabello, S. Villata, and F. Gandon. “Context-aware access control for RDF graph stores”. In: *Frontiers in Artificial Intelligence and Applications*. Vol. 242. IOS Press, 2012, pp. 282–287. ISBN: 9781614990970. DOI: [10.3233/978-1-61499-098-7-282](https://doi.org/10.3233/978-1-61499-098-7-282) (cit. on p. 23).
- [18] L. Costabello et al. “Access control for HTTP operations on linked data”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 7882 LNCS. Springer Verlag, 2013, pp. 185–199. ISBN: 9783642382871. DOI: [10.1007/978-3-642-38288-8_13](https://doi.org/10.1007/978-3-642-38288-8_13) (cit. on p. 23).
- [19] O. Curé and G. Blin. *RDF Database Systems: Triples Storage and SPARQL Query Processing*. en. Morgan Kaufmann, Nov. 2014. ISBN: 9780128004708 (cit. on p. 15).
- [20] R. Curtmola et al. “Searchable symmetric encryption: Improved definitions and efficient constructions”. In: *J. Comput. Secur.* 19.5 (Nov. 2011), pp. 895–934. ISSN: 0926-227X, 1875-8924. DOI: [10.3233/jcs-2011-0426](https://doi.org/10.3233/jcs-2011-0426) (cit. on pp. 21–23).
- [21] C. Dong, G. Russello, and N. Dulay. *Shared and Searchable Encrypted Data for Untrusted Servers*. 2008. DOI: [10.1007/978-3-540-70567-3_10](https://doi.org/10.1007/978-3-540-70567-3_10) (cit. on p. 20).
- [22] T. Elgamal. “A public key cryptosystem and a signature scheme based on discrete logarithms”. In: *IEEE Trans. Inf. Theory* 31.4 (July 1985), pp. 469–472. ISSN: 0018-9448, 1557-9654. DOI: [10.1109/TIT.1985.1057074](https://doi.org/10.1109/TIT.1985.1057074) (cit. on p. 18).
- [23] D. Evans, V. Kolesnikov, and M. Rosulek. “A Pragmatic Introduction to Secure Multi-Party Computation”. In: *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246. ISSN: 2474-1558. DOI: [10.1561/33000000019](https://doi.org/10.1561/33000000019) (cit. on pp. 2, 18).
- [24] B. Ferreira et al. “Boolean Searchable Symmetric Encryption with Filters on Trusted Hardware”. In: *IEEE Trans. Dependable Secure Comput.* (2020), pp. 1–1. ISSN: 1941-0018. DOI: [10.1109/TDSC.2020.3012100](https://doi.org/10.1109/TDSC.2020.3012100) (cit. on pp. 3, 23).
- [25] G. Flouris et al. “Controlling access to RDF graphs”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 6369 LNCS. 2010, pp. 107–117. ISBN: 9783642158766. DOI: [10.1007/978-3-642-15877-3_12](https://doi.org/10.1007/978-3-642-15877-3_12) (cit. on p. 23).

- [26] C. Fontaine and F. Galand. “A survey of homomorphic encryption for nonspecialists”. In: *Eurasip Journal on Information Security* 2007 (2007). ISSN: 2510-523X. DOI: [10.1155/2007/13801](https://doi.org/10.1155/2007/13801) (cit. on pp. 3, 18).
- [27] C. Gentry. “A Fully Homomorphic Encryption Scheme”. PhD thesis. Stanford University, 2009 (cit. on p. 18).
- [28] O. Goldreich. *Foundations of Cryptography: Volume 2, Basic Applications*. en. Cambridge University Press, 2001. ISBN: 9780521830843 (cit. on pp. 2, 18).
- [29] O. Goldreich and R. Ostrovsky. “Software protection and simulation on oblivious RAMs”. In: *J. ACM* 43.3 (May 1996), pp. 431–473. ISSN: 0004-5411. DOI: [10.1145/233551.233553](https://doi.org/10.1145/233551.233553) (cit. on pp. 3, 18).
- [30] M. A. Harris et al. “The Gene Ontology (GO) database and informatics resource”. en. In: *Nucleic Acids Res.* 32.Database issue (Jan. 2004), pp. D258–61. ISSN: 0305-1048, 1362-4962. DOI: [10.1093/nar/gkh036](https://doi.org/10.1093/nar/gkh036) (cit. on p. 1).
- [31] A.-A. Ivan and Y. Dodis. “Proxy Cryptography Revisited”. In: *NDSS*. 2003 (cit. on p. 20).
- [32] A. Jain and C. Farkas. “Secure resource description framework: An access control model”. In: *Proceedings of ACM Symposium on Access Control Models and Technologies, SACMAT*. Vol. 2006. 2006, pp. 121–129. ISBN: 9781595933546 (cit. on p. 23).
- [33] S. Kamara, C. Papamanthou, and T. Roeder. “Dynamic searchable symmetric encryption”. In: *Proceedings of the 2012 ACM conference on Computer and communications security. CCS '12*. Raleigh, North Carolina, USA: Association for Computing Machinery, Oct. 2012, pp. 965–976. ISBN: 9781450316514. DOI: [10.1145/2382196.2382298](https://doi.org/10.1145/2382196.2382298) (cit. on p. 23).
- [34] A. Kasten and A. Scherp. *Iterative Signing of RDF(S) Graphs, Named Graphs, and OWL Graphs: Formalization and Application*. Tech. rep. Mainz: Universität Koblenz-Landau, Mar. 2013 (cit. on p. 23).
- [35] A. Kasten and A. Scherp. “Towards a configurable framework for iterative signing of distributed graph data”. In: *CEUR Workshop Proceedings*. Vol. 1121. 2014 (cit. on p. 23).
- [36] A. Kasten, A. Scherp, and P. Schauß. “A framework for iterative signing of graph data on the web”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 8465 LNCS. Springer Verlag, 2014, pp. 146–160. ISBN: 9783319074429. DOI: [10.1007/978-3-319-07443-6_11](https://doi.org/10.1007/978-3-319-07443-6_11) (cit. on p. 23).
- [37] A. Khaled et al. “A token-based access control system for RDF data in the clouds”. In: *Proceedings - 2nd IEEE International Conference on Cloud Computing Technology and Science, CloudCom 2010*. 2010, pp. 104–111. ISBN: 9780769543024. DOI: [10.1109/CloudCom.2010.76](https://doi.org/10.1109/CloudCom.2010.76) (cit. on p. 23).
- [38] S. Kirrane et al. “Secure manipulation of linked data”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 8218 LNCS. 2013. DOI: [10.1007/978-3-642-41335-3_16](https://doi.org/10.1007/978-3-642-41335-3_16) (cit. on p. 23).
- [39] G. Ladwig. *Efficient Optimization and Processing of Queries Over Text-rich Graph-structured Data*. en. KIT Scientific Publishing, 2013. ISBN: 9783731500155. DOI: [10.5445/KSP/1000034423](https://doi.org/10.5445/KSP/1000034423) (cit. on p. 15).

- [40] A. Letelier et al. “Static analysis and optimization of semantic web queries”. In: *ACM Trans. Database Syst.* 38.4 (Dec. 2013), pp. 1–45. ISSN: 0362-5915. DOI: [10.1145/2500130](https://doi.org/10.1145/2500130) (cit. on p. 24).
- [41] A. Maña, G. Pujol, and C. Pandolfo. “A distributed secure ontology for certified services oriented applications”. In: *Proceedings - 5th IEEE International Conference on Semantic Computing, ICSC 2011*. 2011. DOI: [10.1109/ICSC.2011.105](https://doi.org/10.1109/ICSC.2011.105) (cit. on p. 23).
- [42] J. P. McCrae. *The linked open data cloud*. en. <https://lod-cloud.net/>. Accessed: 2022-2-8 (cit. on p. 2).
- [43] B. D. McKay and A. Piperno. “Practical graph isomorphism, II”. In: *J. Symbolic Comput.* 60 (Jan. 2014), pp. 94–112. ISSN: 0747-7171. DOI: [10.1016/j.jsc.2013.09.003](https://doi.org/10.1016/j.jsc.2013.09.003) (cit. on p. 15).
- [44] G. A. Miller. “WordNet: a lexical database for English”. In: *Commun. ACM* 38.11 (Nov. 1995), pp. 39–41. ISSN: 0001-0782. DOI: [10.1145/219717.219748](https://doi.org/10.1145/219717.219748) (cit. on p. 1).
- [45] G. E. Modoni, M. Sacco, and W. Terkaj. “A survey of RDF store solutions”. In: *2014 International Conference on Engineering, Technology and Innovation: Engineering Responsible Innovation in Products and Services, ICE 2014*. 2014. DOI: [10.1109/ICE.2014.6871541](https://doi.org/10.1109/ICE.2014.6871541) (cit. on p. 15).
- [46] MongoDB, Inc. *MongoDB Manual: Encryption at rest*. en. <https://docs.mongodb.com/manual/core/security-encryption-at-rest/>. Accessed: 2022-2-9 (cit. on p. 3).
- [47] M. A. Musen and Protégé Team. “The Protégé Project: A Look Back and a Look Forward”. en. In: *AI Matters* 1.4 (June 2015), pp. 4–12. ISSN: 2372-3483. DOI: [10.1145/2757001.2757003](https://doi.org/10.1145/2757001.2757003) (cit. on pp. 2, 17).
- [48] R. Neches et al. “Enabling technology for knowledge sharing”. In: *AI Magazine* 12.3 (1991). ISSN: 0738-4602 (cit. on p. 1).
- [49] R. Nejahi, A. Marzouk, and N. Gherabi. “An Enhanced Method for Web Ontologies Encryption”. In: *European Journal of Electrical Engineering and Computer Science* 3.4 (July 2019). ISSN: 2506-9853. DOI: [10.24018/ejece.2019.3.4.99](https://doi.org/10.24018/ejece.2019.3.4.99) (cit. on pp. 23, 24).
- [50] N. Noy et al. “Industry-scale knowledge graphs: Lessons and challenges”. In: *Commun. ACM* 62.8 (2019). ISSN: 0001-0782, 1557-7317. DOI: [10.1145/3331166](https://doi.org/10.1145/3331166) (cit. on p. 1).
- [51] Ontotext. *GraphDB*. en. <https://graphdb.ontotext.com/documentation/>. Accessed: 2022-2-17 (cit. on p. 17).
- [52] OpenLink Software. *Virtuoso Universal Server*. <https://virtuoso.openlinksw.com/>. Accessed: 2022-2-17 (cit. on p. 17).
- [53] Oracle Corporation and/or its affiliates. *MySQL :: MySQL 8.0 reference manual :: 15.13 InnoDB data-at-rest encryption*. en. <https://dev.mysql.com/doc/refman/8.0/en/innodb-data-encryption.html>. Accessed: 2022-2-9 (cit. on p. 3).
- [54] Oracle Corporation and/or its affiliates. *MySQL enterprise encryption*. en. <https://www.mysql.com/products/enterprise/encryption.html>. Accessed: 2022-2-9 (cit. on p. 3).

- [55] owl.cs. *OWL API repository*. en. <https://github.com/owlcs/owlapi>. Accessed: 2022-2-17 (cit. on p. 18).
- [56] P. Paillier. “Public-Key Cryptosystems Based on Composite Degree Residuosity Classes”. In: *Advances in Cryptology — EUROCRYPT ’99*. Springer Berlin Heidelberg, 1999, pp. 223–238. DOI: [10.1007/3-540-48910-X_16](https://doi.org/10.1007/3-540-48910-X_16) (cit. on p. 18).
- [57] V. Papakonstantinou et al. “Access control for RDF graphs using abstract models”. In: *Proceedings of ACM Symposium on Access Control Models and Technologies, SACMAT*. 2012. DOI: [10.1145/2295136.2295155](https://doi.org/10.1145/2295136.2295155) (cit. on p. 23).
- [58] W. Pearson. *Cryptography and Network Security: Principles and Practice, Global Edition*. 7th ed. Pearson Education, 2017. ISBN: 9781292158587 (cit. on pp. 3, 21).
- [59] Z. Peng, S. Tang, and L. Jiang. *A Symmetric Authenticated Proxy Re-encryption Scheme with Provable Security*. 2017. DOI: [10.1007/978-3-319-68542-7_8](https://doi.org/10.1007/978-3-319-68542-7_8) (cit. on p. 20).
- [60] J. Pérez, M. Arenas, and C. Gutierrez. “Semantics and Complexity of SPARQL”. In: *Semantics and complexity of SPARQL*. *ACM Trans. Database Syst* 34.16 (2009). DOI: [10.1145/1567274.1567278](https://doi.org/10.1145/1567274.1567278) (cit. on p. 24).
- [61] F. Picalausa and S. Vansummeren. “What are real SPARQL queries like?” In: *Proceedings of the International Workshop on Semantic Web Information Management, SWIM ’11 Article 7*. Athens, Greece: Association for Computing Machinery, June 2011, pp. 1–6. ISBN: 9781450306515. DOI: [10.1145/1999299.1999306](https://doi.org/10.1145/1999299.1999306) (cit. on pp. 16, 24).
- [62] G. S. Poh, M. S. Mohamad, and M. R. Z’Aba. “Structured encryption for conceptual graphs”. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 7631 LNCS. Springer Verlag, 2012, pp. 105–122. ISBN: 9783642341168. DOI: [10.1007/978-3-642-34117-5_7](https://doi.org/10.1007/978-3-642-34117-5_7) (cit. on pp. 4, 23, 24).
- [63] R. A. Popa, N. Zeldovich, and H. Balakrishnan. “CryptDB : A Practical Encrypted Relational DBMS”. In: *Designfax* (2011). ISSN: 0163-6669 (cit. on p. 23).
- [64] R. A. Popa et al. “CryptDB: Processing queries on an encrypted database”. In: *Communications of the ACM*. Vol. 55. 2012. DOI: [10.1145/2330667.2330691](https://doi.org/10.1145/2330667.2330691) (cit. on p. 23).
- [65] R. A. Popa et al. “CryptDB: Protecting confidentiality with encrypted query processing”. In: *SOSP’11 - Proceedings of the 23rd ACM Symposium on Operating Systems Principles*. 2011. DOI: [10.1145/2043556.2043566](https://doi.org/10.1145/2043556.2043566) (cit. on p. 23).
- [66] R. L. Rivest, L. Adleman, and M. L. Dertouzos. “On data banks and privacy homomorphisms”. In: *Foundations of secure* (1978) (cit. on p. 18).
- [67] R. L. Rivest, A. Shamir, and L. Adleman. “A method for obtaining digital signatures and public-key cryptosystems”. en. In: *Commun. ACM* 21.2 (Feb. 1978), pp. 120–126. ISSN: 0001-0782, 1557-7317. DOI: [10.1145/359340.359342](https://doi.org/10.1145/359340.359342) (cit. on p. 18).
- [68] K. Sakurai, T. Nishide, and A. Syalim. *Improved proxy re-encryption scheme for symmetric key cryptography*. 2017. DOI: [10.1109/iwbis.2017.8275110](https://doi.org/10.1109/iwbis.2017.8275110) (cit. on p. 20).

- [69] Security Compass. *The 2021 State of Cloud Adoption*. en. <https://resources.securitycompass.com/reports/2021-state-of-cloud-adoption>. Accessed: 2022-2-9. Sept. 2021 (cit. on p. 2).
- [70] N. Sioutos et al. “NCI Thesaurus: a semantic model integrating cancer-related clinical and molecular information”. en. In: *J. Biomed. Inform.* 40.1 (Feb. 2007), pp. 30–43. ISSN: 1532-0464, 1532-0480. DOI: [10.1016/j.jbi.2006.02.013](https://doi.org/10.1016/j.jbi.2006.02.013) (cit. on p. 1).
- [71] E. Sirin et al. “Pellet: A practical OWL-DL reasoner”. In: *Journal of Web Semantics* 5.2 (June 2007), pp. 51–53. ISSN: 1570-8268. DOI: [10.1016/j.websem.2007.03.004](https://doi.org/10.1016/j.websem.2007.03.004) (cit. on p. 17).
- [72] D. X. Song, D. Wagner, and A. Perrig. “Practical techniques for searches on encrypted data”. In: *Proceeding 2000 IEEE Symposium on Security and Privacy. S P 2000*. May 2000, pp. 44–55. DOI: [10.1109/SECPRI.2000.848445](https://doi.org/10.1109/SECPRI.2000.848445) (cit. on pp. 3, 19).
- [73] D. X. Song, D. Wagner, and A. Perrig. “Practical techniques for searches on encrypted data”. In: *Proceeding 2000 IEEE Symposium on Security and Privacy. S P 2000*. May 2000, pp. 44–55. DOI: [10.1109/SECPRI.2000.848445](https://doi.org/10.1109/SECPRI.2000.848445) (cit. on p. 21).
- [74] R. Studer, V. R. Benjamins, and D. Fensel. “Knowledge engineering: Principles and methods”. In: *Data Knowl. Eng.* 25.1 (Mar. 1998), pp. 161–197. ISSN: 0169-023X. DOI: [10.1016/S0169-023X\(97\)00056-6](https://doi.org/10.1016/S0169-023X(97)00056-6) (cit. on p. 10).
- [75] G. Suntaxi, A. A. El Ghazi, and K. Böhm. “Secrecy and performance models for query processing on outsourced graph data”. In: *Distributed and Parallel Databases* 39.1 (Mar. 2021), pp. 35–77. ISSN: 1573-7578. DOI: [10.1007/s10619-020-07284-0](https://doi.org/10.1007/s10619-020-07284-0) (cit. on pp. 23, 24).
- [76] A. Sutton and R. Samavi. “Timestamp-based Integrity Proofs for Linked Data”. In: *Proceedings of the International Workshop on Semantic Big Data, SBD 2018 - In conjunction with the 2018 ACM SIGMOD/PODS Conference*. Association for Computing Machinery, Inc, June 2018. ISBN: 9781450357791. DOI: [10.1145/3208352.3208353](https://doi.org/10.1145/3208352.3208353) (cit. on p. 23).
- [77] A. Syalim, T. Nishide, and K. Sakurai. “Realizing Proxy Re-encryption in the Symmetric World”. In: *Informatics Engineering and Information Science*. Springer Berlin Heidelberg, 2011, pp. 259–274. DOI: [10.1007/978-3-642-25327-0_23](https://doi.org/10.1007/978-3-642-25327-0_23) (cit. on p. 20).
- [78] The Apache Software Foundation. *Apache Jena*. en. <https://jena.apache.org/>. Accessed: 2022-2-17 (cit. on p. 18).
- [79] The Apache Software Foundation. *Apache Jena - TDB*. en. <https://jena.apache.org/documentation/tdb/index.html>. Accessed: 2022-2-17 (cit. on p. 17).
- [80] T. Tudorache, J. Vendetti, and N. F. Noy. “Web-Protégé: A lightweight OWL ontology editor for the web”. In: *CEUR Workshop Proceedings*. Vol. 432. 2009 (cit. on pp. 2, 17).
- [81] T. Tudorache et al. “Supporting collaborative ontology development in Protégé”. In: *Lect. Notes Comput. Sci.* 5318 LNCS (2008), pp. 17–32. ISSN: 0302-9743, 1611-3349. DOI: [10.1007/978-3-540-88564-1_2](https://doi.org/10.1007/978-3-540-88564-1_2) (cit. on pp. 2, 17).
- [82] G. Tummarello et al. “Signing individual fragments of an RDF graph”. In: *14th International World Wide Web Conference, WWW2005*. 2005, pp. 1020–1021. ISBN: 9781595930514. DOI: [10.1145/1062745.1062848](https://doi.org/10.1145/1062745.1062848) (cit. on p. 23).

-
- [83] W3C. *Extensible markup language (XML) 1.0 (fifth edition)*. en. <https://www.w3.org/TR/xml/>. Accessed: 2022-2-11 (cit. on p. 7).
 - [84] W3C. *OWL 2 web ontology language direct semantics (second edition)*. en. <https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/>. Accessed: 2022-2-14 (cit. on pp. 10, 11).
 - [85] W3C. *OWL 2 web ontology language document overview (second edition)*. en. <https://www.w3.org/TR/owl2-overview/>. Accessed: 2022-2-11 (cit. on pp. 7, 10, 11).
 - [86] W3C. *OWL 2 web ontology language Manchester syntax (second edition)*. en. <https://www.w3.org/TR/2012/NOTE-owl2-manchester-syntax-20121211/>. Accessed: 2022-2-14 (cit. on p. 11).
 - [87] W3C. *OWL 2 web ontology language profiles (second edition)*. en. <https://www.w3.org/TR/2012/REC-owl2-profiles-20121211/>. Accessed: 2022-2-14 (cit. on p. 11).
 - [88] W3C. *OWL 2 web ontology language quick reference guide (second edition)*. en. <https://www.w3.org/TR/owl2-quick-reference/>. Accessed: 2022-2-14 (cit. on p. 11).
 - [89] W3C. *OWL 2 web ontology language RDF-based semantics (second edition)*. en. <https://www.w3.org/TR/2012/REC-owl2-rdf-based-semantics-20121211/>. Accessed: 2022-2-14 (cit. on p. 10).
 - [90] W3C. *OWL 2 web ontology language structural specification and functional-style syntax (second edition)*. en. <https://www.w3.org/TR/2012/REC-owl2-syntax-20121211/>. Accessed: 2022-2-14 (cit. on p. 11).
 - [91] W3C. *OWL 2 web ontology language XML serialization (second edition)*. en. <https://www.w3.org/TR/2012/REC-owl2-xml-serialization-20121211/>. Accessed: 2022-2-14 (cit. on p. 11).
 - [92] W3C. *RDF 1.1 concepts and abstract syntax*. en. <https://www.w3.org/TR/rdf11-concepts/>. Accessed: 2022-2-11 (cit. on pp. 7, 8).
 - [93] W3C. *RDF 1.1 TriG*. en. <https://www.w3.org/TR/trig/>. Accessed: 2022-2-13 (cit. on p. 8).
 - [94] W3C. *RDF 1.1 turtle*. en. <https://www.w3.org/TR/turtle/>. Accessed: 2022-2-13 (cit. on p. 8).
 - [95] W3C. *RDF 1.1 XML syntax*. en. <https://www.w3.org/TR/rdf-syntax-grammar/>. Accessed: 2022-2-13 (cit. on p. 9).
 - [96] W3C. *RDF Schema 1.1*. en. <https://www.w3.org/TR/rdf-schema/>. Accessed: 2022-2-11 (cit. on pp. 7, 9).
 - [97] W3C. *RDFa core 1.1 - third edition*. en. <https://www.w3.org/TR/rdfa-core/>. Accessed: 2022-2-13 (cit. on p. 9).
 - [98] W3C. *Semantic Web Case Studies and Use Cases*. en. <https://www.w3.org/2001/sw/sweo/public/UseCases/>. Accessed: 2022-2-8 (cit. on p. 1).
 - [99] W3C. *SPARQL 1.1 graph store HTTP protocol*. en. <https://www.w3.org/TR/sparql11-http-rdf-update/>. Accessed: 2022-2-4. Mar. 2013 (cit. on p. 15).
 - [100] W3C. *SPARQL 1.1 Overview*. <https://www.w3.org/TR/sparql11-overview/>. Mar. 2013 (cit. on pp. 4, 15).

BIBLIOGRAPHY

- [101] W3C. *SPARQL 1.1 update*. en. <https://www.w3.org/TR/sparql11-update/>. Accessed: 2022-2-15 (cit. on p. 15).
- [102] W3C. *SPARQL Query Language for RDF*. <https://www.w3.org/TR/rdf-sparql-query/>. Jan. 2008 (cit. on pp. 4, 15, 16).
- [103] W3C. *SWRL: A Semantic Web Rule Language Combining OWL and RuleML*. <https://www.w3.org/Submission/SWRL/>. May 2004 (cit. on p. 7).
- [104] Wikipedia contributors. *Semantic web stack*. https://en.wikipedia.org/wiki/File:Semantic_web_stack.svg. Accessed: 2022-2-15 (cit. on p. 7).